

4. Write a program to implement loss functions for neural network.

```

import numpy as np

# -----
# 1. Mean Squared Error (MSE)
# -----
def mean_squared_error(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

# -----
# 2. Mean Absolute Error (MAE)
# -----
def mean_absolute_error(y_true, y_pred):
    return np.mean(np.abs(y_true - y_pred))

# -----
# 3. Binary Cross Entropy (BCE)
# -----
def binary_cross_entropy(y_true, y_pred):
    epsilon = 1e-15 # To avoid log(0)

    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)

    return -np.mean(
        y_true * np.log(y_pred) +
        (1 - y_true) * np.log(1 - y_pred)
    )

# -----
# 4. Categorical Cross Entropy (CCE)
# -----
def categorical_cross_entropy(y_true, y_pred):
    epsilon = 1e-15 # To avoid log(0)

    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)

    return -np.sum(y_true * np.log(y_pred)) / y_true.shape[0]

# -----
# Testing the Loss Functions
# -----

# Regression Example
y_true_reg = np.array([1.0, 2.0, 3.0])

```

```
y_pred_reg = np.array([0.8, 2.2, 2.9])

print("MSE:", mean_squared_error(y_true_reg, y_pred_reg))
print("MAE:", mean_absolute_error(y_true_reg, y_pred_reg))

# Binary Classification Example
y_true_bin = np.array([1, 0, 1, 0])
y_pred_bin = np.array([0.9, 0.2, 0.8, 0.1])

print("\nBinary Cross Entropy:",
      binary_cross_entropy(y_true_bin, y_pred_bin))

# Multi-class Classification Example (One-hot Encoded)
y_true_cat = np.array([
    [1, 0, 0],
    [0, 1, 0],
    [0, 0, 1]
])

y_pred_cat = np.array([
    [0.8, 0.1, 0.1],
    [0.2, 0.7, 0.1],
    [0.1, 0.2, 0.7]
])

print("Categorical Cross Entropy:",
      categorical_crossentropy(y_true_cat, y_pred_cat))
```